

VIVA · INTERVIEW · DEMONSTRATION

IntelliQR Questions & Answers

A structured preparation bank covering concept, architecture, implementation, security, deployment, testing, troubleshooting and practical demonstration.

Includes 90 model questions with concise answers

How to use this guide

Learn the ideas rather than memorizing sentences. A strong answer normally follows: **purpose** → **implementation** → **reason** → **limitation**. When asked to demonstrate, show the deployed behavior and then connect it to the responsible frontend, endpoint, service, repository and storage component.

Thirty-second project introduction: “IntelliQR is a full-stack QR management platform built with React, TypeScript, FastAPI and MongoDB. It supports seven QR payload types, BrandCraft customization, dynamic tracking and analytics, SafeScan decoding, SecureVault URL protection, encrypted ShareVault files, BulkForge batch jobs, notifications and admin controls. It is deployed with Vercel, Render, MongoDB Atlas, private Vercel Blob and Brevo’s HTTPS email API.”

Recommended demonstration order

1. Login and refresh the page to show session restoration.
2. Create and download a QR with BrandCraft settings.
3. Scan a dynamic QR on a phone and show analytics.
4. Create a ShareVault file, scan its QR, download and show notification/email.
5. Show SafeScan, BulkForge, profile and admin boundaries.

A. Project overview

1. What problem does IntelliQR solve?

It centralizes QR creation, customization, management, tracking, security and file sharing. A normal QR generator only creates an image; IntelliQR persists records, enables editable redirects, analytics, controlled access, batch generation and notifications.

2. Who are the intended users?

Individuals, small teams, educators, marketers or businesses that need branded QR codes, scan insight, protected links, secure file distribution or bulk exports.

3. What are the main modules?

Authentication, Dashboard, QR Generator/BrandCraft, Scanner/SafeScan, Analytics, SecureVault, ShareVault, BulkForge, Profile, Notifications and Administration.

4. Is IntelliQR only a frontend project?

No. It is full stack: React UI, FastAPI REST API, MongoDB persistence, Vercel Blob object storage and Brevo email delivery.

5. What makes the project technically significant?

It combines typed frontend/backend contracts, QR standards, dynamic redirects, privacy-aware analytics, JWT rotation, AES-GCM encryption, durable cloud storage, role-based access and automated tests.

6. What is the difference between SecureVault and ShareVault?

SecureVault protects a URL destination behind a dynamic QR. ShareVault encrypts and distributes an uploaded file. Both support policies, but they secure different resources.

B. Architecture and stack

7. Explain the production architecture.

Vercel hosts the React SPA and rewrites API traffic to FastAPI on Render. Render stores metadata in MongoDB Atlas, durable private files in Vercel Blob and sends transactional email through Brevo's HTTPS API.

8. Why React and TypeScript?

React supports reusable stateful components. TypeScript detects mismatched API contracts and component props before runtime, improving maintainability.

9. Why Vite?

It provides a fast development server, TypeScript integration and an optimized production build with minimal configuration.

10. Why FastAPI?

FastAPI combines Python productivity, async support, dependency injection, Pydantic validation and automatic OpenAPI/Swagger documentation.

11. Why MongoDB?

The project contains varied nested documents such as design objects, QR details, policies and analytics. MongoDB provides flexible documents, indexes and aggregation while Pydantic preserves boundary validation.

12. Why not store production files on Render?

Free Render's filesystem is ephemeral. Redeployment can erase generated files. Private Vercel Blob is used for durable ShareVault and QR artifacts.

13. Why use Brevo API instead of SMTP?

Free Render blocks outbound SMTP ports 25, 465 and 587. Brevo's HTTPS API works over normal web traffic and provides a free daily allowance.

14. What is dependency injection in this backend?

`dependencies.py` constructs repositories, services and infrastructure adapters for endpoints. It reduces coupling and makes implementations replaceable and testable.

15. Describe the backend layers.

Endpoints handle HTTP; schemas validate boundaries; services enforce business rules; repositories handle MongoDB; models represent domain state; infrastructure integrates QR, files, ZIP and email.

C. Frontend design

16. How are frontend features organized?

Each feature has typed API functions, hooks for state/orchestration, types and optional components. Pages compose features, and shared UI/layout components provide consistent presentation.

17. What does `ProtectedRoute` do?

It waits for authentication initialization, blocks anonymous access and optionally enforces a required role such as admin.

18. How was the refresh-to-registration problem solved?

The app restores the session using the `HttpOnly` refresh cookie before routing. `ProtectedRoute` shows a loading state until initialization finishes instead of prematurely redirecting.

19. What does the shared API client do?

It builds API URLs, adds credentials and Bearer tokens, parses typed errors, downloads blobs and performs one refresh-and-retry when an authenticated request returns 401.

20. Why lazy-load pages?

React lazy loading splits large routes into smaller chunks, reducing initial download and loading features only when visited.

21. How are dark and light themes implemented?

`useTheme` applies a theme attribute to the HTML element. Global CSS defines alternate colors/surfaces, while reusable components inherit the system.

22. How does internationalization work?

`LocaleContext` supplies translation dictionaries for English, Hindi and Gujarati. The selected locale is saved to the account and restored with the session.

23. What is the Online/Offline indicator?

A shared hook calls the backend health endpoint and renders status in the top bar. It reflects API reachability and can briefly be offline during Render cold start.

D. Authentication and authorization

24. How are passwords stored?

As bcrypt hashes through pwlib. Plain passwords are never saved.

25. Why use access and refresh tokens?

A short-lived access token limits exposure, while the longer refresh cookie restores sessions without keeping a long-lived readable token in JavaScript.

26. Where is the access token stored?

Only in application memory, not localStorage. This reduces persistent token theft through XSS.

27. Why is the refresh cookie HttpOnly?

Browser JavaScript cannot read HttpOnly cookies, so an injected script cannot directly steal the refresh token.

28. What is refresh-token rotation?

Each successful refresh revokes/replaces the previous refresh session and issues a new token, limiting replay of an older token.

29. How does password change sign out all devices?

It updates the hash, increments the user token version and revokes all refresh sessions. Existing tokens no longer match current authorization state.

30. How is admin access assigned?

Emails in the server-side `INTELLIQR_ADMIN_EMAILS` allowlist receive or synchronize the admin role. The API enforces role checks.

31. Why is hiding the Admin link insufficient?

A user can manually call an endpoint. Real security must be enforced on the backend; UI hiding is only presentation.

E. QR generation and BrandCraft

32. Which QR types are supported?

URL, text, email, phone, Wi-Fi, contact/vCard and geographic location.

33. How is a QR generated?

The request is validated, PayloadBuilder creates standard content, Renderer builds one matrix and renders PNG/SVG/PDF, Blob stores the files, and MongoDB stores metadata and paths.

34. What does BrandCraft customize?

Foreground/background colors, gradient/direction, square/rounded/dot modules, frame style/text, error correction, size, margin and optional logo.

35. Why validate color contrast?

Low contrast makes QR modules difficult for cameras to distinguish. A minimum contrast rule protects scan reliability.

36. What is error correction?

Levels L, M, Q and H add redundancy so a QR can survive increasing damage/obstruction. Higher correction reduces available payload capacity.

37. Why is a quiet zone important?

The blank margin separates the QR from surrounding content and helps detectors locate its boundary.

38. How is a logo made safer?

Only validated PNG/JPEG data is accepted, dimensions/bytes are limited, it is normalized to PNG and placed over a background plate at a limited proportion of QR size.

39. Why generate SVG and PDF as well as PNG?

PNG is convenient for screens, SVG scales without pixelation, and PDF is suitable for print/document workflows.

40. Why did old downloads return Not Found?

They referenced Render's temporary filesystem and were erased by deployment. New files use private Vercel Blob; lost old artifacts must be regenerated.

F. Dynamic QR and analytics

41. What is a dynamic QR?

It encodes a stable backend redirect URL rather than the final destination. The backend can update the destination and record scans.

42. What is a direct/static QR?

It encodes the final payload. It continues working without the backend but cannot provide redirect tracking or destination editing.

43. Describe a tracked scan.

The phone requests `/r/{slug}`, the backend validates status/limits, atomically consumes a scan, stores analytics context and returns a 307 redirect.

44. Why use an atomic scan update?

Concurrent scans must not exceed limits or overwrite counts. An atomic database operation checks state and increments as one operation.

45. What analytics are shown?

Totals, unique visitor estimates, trends, devices, browsers, operating systems, locality, top QR codes and recent scans.

46. How is visitor privacy protected?

A keyed one-way hash estimates repeat visitors. Raw IP and full user-agent are not retained.

47. Why can location show Unknown?

The app avoids sending IP addresses to an external geolocation service. Accurate public location requires an optional local GeoIP database.

48. Why can tracking fail after deployment?

Common causes are creating a static QR, using an old image, or a misspelled `INTELLIQR_REDIRECT_BASE_URL`. Decode the actual QR and verify the hostname.

G. Scanner and SafeScan

49. How are QR images decoded?

The backend validates uploaded image bytes and uses OpenCV/pyzbar to detect one or more QR symbols.

50. Are uploaded scan images stored?

No. Only decoded content and analysis metadata are persisted.

51. What does SmartClassifier do?

It interprets decoded syntax and classifies website, email, contact, phone, Wi-Fi, payment or text content.

52. What does SafeScan do?

It applies transparent local heuristics to URL content and reports suspicious characteristics and risk information.

53. Can SafeScan guarantee safety?

No. It is a heuristic aid, not an antivirus engine or remote sandbox. False positives and negatives are possible.

H. SecureVault and ShareVault

54. Which access policies exist?

Public, password, authenticated members and private email allowlist.

55. How are protected destinations stored?

Encrypted with AES-256-GCM using a stable server-side key and unique nonce.

56. Why AES-GCM?

It provides confidentiality and authentication, detecting tampering as well as hiding content.

57. What is a Vault grant?

A short-lived JWT bound to a slug, purpose and visitor/user. It proves policy approval without exposing stored secrets.

58. Why are grant purposes separated?

A ShareVault grant must not be reusable as a SecureVault redirect grant. Purpose binding prevents cross-feature token confusion.

59. How are ShareVault files validated?

The service checks byte limit, approved extension and key signatures/container markers. Images are decoded and dimension-limited.

60. Where are ShareVault files stored?

Encrypted bytes are stored in private Vercel Blob. MongoDB stores metadata and the object path.

61. Are files decrypted on Vercel?

No. The FastAPI backend fetches encrypted bytes and decrypts them in memory only after a valid grant.

62. Why does a ShareVault QR point to Vercel directly?

It opens the public access UI at `/share/:slug`. ShareVault records analytics on authorized download, so an extra dynamic redirect is unnecessary.

63. What happens when a share reaches its limit?

The atomic consume operation makes it unavailable, records the final event and can send a limit notification.

64. What happens when the Vault encryption key changes?

Existing encrypted destinations/files cannot be decrypted. The key must be backed up and kept stable.

I. BulkForge, dashboard and admin

65. How does BulkForge work?

It parses CSV/XLSX, validates rows, reuses the QR generator, tracks progress/errors and packages selected formats into a ZIP.

66. Why isolate row errors?

One invalid row should not discard all valid rows. Completed jobs can be partial with bounded error details.

67. What are BulkForge states?

Queued, processing, completed, partial and failed.

68. What does the Dashboard summarize?

QRs/favorites, redirect scans, scanner history, shares/downloads, exports/jobs, recent activity and download history.

69. What safeguards exist in Admin controls?

Role enforcement, audit events, session invalidation, prohibition on self-access changes and protection of the final active administrator.

70. Why record admin audits?

They provide accountability: who changed which target, what action occurred and when.

J. Notifications and email

71. Which events create notifications?

Risky scans, dynamic QR scan-limit events, ShareVault downloads/limits and BulkForge completion/failure.

72. Can users control notifications?

Yes. They control in-app/email delivery and category preferences for security, QR, ShareVault and BulkForge.

73. How is email delivered?

FastAPI sends a POST request to Brevo's transactional email HTTPS endpoint using a hidden API key.

74. How is the email designed?

It contains responsive branded HTML, an event card, IntelliQR call-to-action, security note and plain-text fallback.

75. How is HTML injection prevented?

Notification title and message are HTML-escaped before being inserted into the template.

76. What is the limitation of a Gmail Brevo sender?

The project does not control gmail.com DNS, so it cannot configure custom DKIM/DMARC. A custom owned domain is better for production deliverability.

K. Deployment and environment

77. What are the production URLs?

Frontend: `https://intelij-qr.vercel.app`. Backend: `https://intelijqr-backend.onrender.com`.

78. Why is exact hostname spelling important?

Dynamic QR images permanently encode the configured base URL. A single extra letter produces a different host and 404; existing images cannot change afterward.

79. What does `vercel.json` do?

It rewrites `/api/*` to Render and all other application routes to `index.html` for React Router refresh support.

80. What production secrets are required?

MongoDB URI, JWT secret, Vault key, Blob token and Brevo key, plus safe URLs/origins and sender configuration.

81. Why should secrets not be committed?

Anyone with repository access could use the database, forge tokens, decrypt content, modify blobs or send email. Secrets belong in platform environment settings.

82. What is a Render cold start?

A free inactive service sleeps. The first request wakes it and can take around 50 seconds, temporarily showing the backend as offline.

L. Testing and troubleshooting

83. What backend tests exist?

pytest covers security, schemas, admin, profile/dashboard, analytics, QR generation/storage, scanning, SecureVault, ShareVault, BulkForge, notifications/Brevo and API contracts.

84. How is frontend quality checked?

TypeScript type checking, Vitest API tests and a Vite production build.

85. What command starts the backend?

`.\.venv\Scripts\python.exe -m uvicorn app.main:app --host 0.0.0.0 --port 8000` from the backend directory.

86. Which Python file is the backend entry point?

backend/app/main.py ; Uvicorn imports the `app` `app.main:app`.

87. Why create `.venv` ?

It isolates project packages and versions from system Python and other projects, enabling repeatable development.

88. How would you diagnose a QR opening Not Found?

Decode the actual image, inspect its URL, test the hostname/path, verify Render environment variables, confirm the newest deploy and regenerate the QR after correction.

89. How would you diagnose an empty Blob store?

Confirm the latest code deployed, verify the exact prefixed environment key, create a new upload, search the expected prefix and check backend logs for upload errors.

90. What would you improve next?

Durable BulkForge ZIPs, queue workers, rate limiting, MFA/CAPTCHA, local GeoIP, object reconciliation, custom email domain and full browser end-to-end tests.

Rapid-fire revision

Prompt	One-line response
Frontend	React 19 + TypeScript + Vite + Tailwind.
Backend	FastAPI + Uvicorn + Pydantic.
Database	MongoDB Atlas through async PyMongo.
Object storage	Private Vercel Blob.
Email	Brevo transactional HTTPS API.
Password security	Bcrypt hashes.
Encryption	AES-256-GCM.
Sessions	Memory access JWT + rotated HttpOnly refresh cookie.
Dynamic route	<code>/r/{slug}</code> .
Public ShareVault route	<code>/share/:slug</code> .
Backend entry	<code>backend/app/main.py</code> .
Frontend entry	<code>frontend/src/main.tsx</code> and <code>App.tsx</code> .
Backend tests	<code>python -m pytest</code> .
Frontend checks	<code>npm run typecheck</code> , <code>npm test</code> , <code>npm run build</code> .

Do not claim: perfect malware detection, permanent free Render availability, precise location without GeoIP, durable old artifacts that were already deleted, or custom-domain email authentication when using Gmail.

End of IntelliQR Viva Questions and Answers.